Supplementary Requirements Documentation

Subgroup 3: Devices

Revision History

Date	Version	Description	Author
2026-02-06	1.0	Initial revision	Sergej Macut
2026-02-26	1.1	Revised	Sergej Macut
2026-02-27	1.2	Added S5 Fault Handling supp. Req.	Patrick Nordahl
2026-04-23	1.3	Autonomous Safety & Environmental Responsiveness	Mustafa Al-Bayati
2026-05-08	1.4	Refactored architecture to support dual-mode operation: manual user interaction via Node.js gateway and autonomous response via Observer design pattern.	Mustafa Al-Bayati, Ryad Hazin, Tony Nguyen, Sergej Macut
2026-05-08	1.5	Updated file to contain Table of content, figures and references	Mustafa Al-Bayati, Ryad Hazin, Tony Nguyen, Sergej Macut

Supplementary Requirements List

Supplementary Requirement Name	Priority
S1. Interoperability - Middleman Architecture	Essential
S2.Performance - Response Latency	Desirable
S3. Reliability - Simulation Stability	Essential
S4. Design Language - UML Standards	Essential
S5. Fault Handling & State Recovery	Essential
S6. Autonomous Safety & Environmental Responsiveness	Essential

Supplementary Requirements Descriptions

S1. Interoperability - Middleman Architecture

Devices must not interact with user units directly but instead communicate through an application acting as a middleman. The system is designed to handle two simultaneous interaction flows:

1. **User Interaction:** Manual commands are issued from the UI, processed by the **Node.js gateway**, and forwarded to the device for execution.
2. **Autonomous Interaction:** Sensor driven events are managed internally using the **Observer Design Pattern**, allowing local device coordination without requiring server communication.

S2. Performance - Response Latency

The device state must update within the system quickly, less than 1000 ms to support fluid human-controlled activities from a mobile or web interface [1]. Any physical hardware must process incoming server commands without noticeable delay to ensure a responsive user experience.

S3. Reliability - Simulation Stability

Devices must be capable of running continuously on Arduino without crashing. The software must maintain a consistent state even when multiple communicating units are observing the device simultaneously through the server.

S4. Design Language - UML Standards

The device architecture must be modeled using standardized UML diagrams to ensure technical clarity across all project subgroups. This includes the use of class diagrams for structure and interaction sequence diagrams to map how devices talk to the server.

S5. Fault Handling – Communication Robustness

The device software shall handle communication errors gracefully. If serial communication is interrupted or corrupted, the device must:

Continue to operate in a safe local state

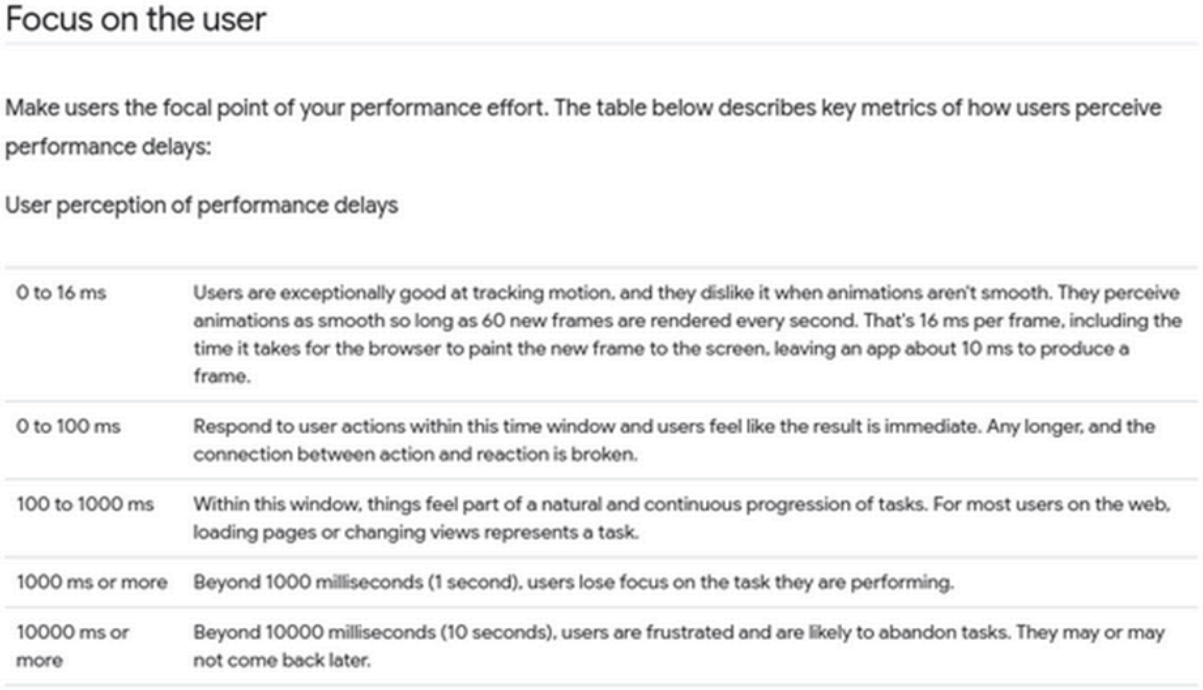
Avoid freezing or crashing

Maintain the last known valid device state.

S6. Autonomous Safety & Environmental Responsiveness

- The device must be capable of reacting to environmental conditions **independently of external systems** (Gateway or Server). By implementing the **Observer Design Pattern**, safety-critical components like `GasSafety` or `SteamSensor` (Subjects) can notify their respective actuators (Observers) directly. This ensures that while **user interaction** is available via the gateway for general control, the house maintains its ability to protect itself from hazards even during communication failures.

Figures



Focus on the user

Make users the focal point of your performance effort. The table below describes key metrics of how users perceive performance delays:

User perception of performance delays

0 to 16 ms	Users are exceptionally good at tracking motion, and they dislike it when animations aren't smooth. They perceive animations as smooth so long as 60 new frames are rendered every second. That's 16 ms per frame, including the time it takes for the browser to paint the new frame to the screen, leaving an app about 10 ms to produce a frame.
0 to 100 ms	Respond to user actions within this time window and users feel like the result is immediate. Any longer, and the connection between action and reaction is broken.
100 to 1000 ms	Within this window, things feel part of a natural and continuous progression of tasks. For most users on the web, loading pages or changing views represents a task.
1000 ms or more	Beyond 1000 milliseconds (1 second), users lose focus on the task they are performing.
10000 ms or more	Beyond 10000 milliseconds (10 seconds), users are frustrated and are likely to abandon tasks. They may or may not come back later.

Figure 1: User perception of performance delays [1]

References:

[1] “Measure performance with the RAIL model | Articles,” web.dev. Accessed: Apr. 21, 2026. [Online]. Available: https://web.dev/articles/rail